

TP Raspberry Pi

C, C++, Python ...

Prof : **Kamal Boudjelaba**

15 septembre 2026

Table des matières

1	Utilisation des broches GPIO du Raspberry Pi 5	3
1.1	Numérotation des broches	3
1.2	Câblage de la LED	3
1.3	Vérifier le fonctionnement du GPIO	4
2	Commander une LED avec Python	5
2.1	La bibliothèque GPIO Zero	5
2.2	Allumer une LED	5
2.3	Éteindre la LED	5
2.4	Faire clignoter la LED	5
2.5	Analyse du programme	6
2.6	Questions	6
3	Variation de l'intensité lumineuse	7
3.1	Programme	7
3.2	Travail demandé	7
4	Commander une LED depuis un navigateur	8
4.1	Principe	8
4.2	Installation de Flask	8
4.3	Créer le serveur Web	9
4.4	Lancer le serveur	10
4.5	Tester depuis le Raspberry Pi	10
4.6	Tester depuis un autre appareil	10
4.7	Arrêter le serveur	11
5	Travaux Pratiques	12
5.1	TP 1 : Commander une LED depuis un navigateur	12
5.2	Amélioration 1 : ajouter un bouton de clignotement	12
5.3	Amélioration 2 : modifier le temps de clignotement	12
5.4	Amélioration 3 : afficher l'état de la LED	12
6	Questions de synthèse	13

Préambule

Prérequis

- Maîtrise de base du Raspberry Pi et de son système Linux.
- Connaissances élémentaires des commandes du terminal.
- Notions de base en électronique : LED, résistance, alimentation, masse.
- Notions élémentaires de programmation en Python.

Matériel

- Un Raspberry Pi 5.
- Raspberry Pi OS.
- Une plaque d'expérimentation (breadboard).
- Une LED.
- Une résistance de limitation de courant, par exemple 220 Ω .
- Des fils de connexion.

Objectifs

- Identifier et utiliser une broche GPIO du Raspberry Pi 5.
- Commander une sortie GPIO à partir d'un programme Python.
- Utiliser la bibliothèque GPIO Zero.
- Réaliser un programme permettant de faire clignoter une LED.
- Mettre en œuvre un serveur Web avec Flask.
- Commander une sortie GPIO depuis un navigateur Web.

Compétences visées

- **C08** : Coder
- **C09** : Installer un réseau informatique
- **C10** : Exploiter un réseau informatique

Activités et tâches associées

- **R2 – Installation et qualification**
 - R2-T5 : Réalisation des opérations avec contrôle matériel et logiciel
- **R3 – Exploitation et maintien en condition opérationnelle**
 - R3-T3 : Supervision de l'état du réseau
 - R3-T5 : Configuration matérielle et logicielle des équipements
- **D2 – Développement et validation de solutions logicielles**
 - D2-T3 : Développement, utilisation ou adaptation de composants logiciels

1. Utilisation des broches GPIO du Raspberry Pi 5

Le Raspberry Pi 5 possède un connecteur GPIO à 40 broches permettant d'interagir avec différents composants électroniques.

Dans ce TP, nous utiliserons une broche GPIO comme sortie numérique afin de commander une LED.

1.1 Numérotation des broches

Dans les programmes Python utilisant GPIO Zero, les broches sont généralement désignées par leur numéro **GPIO**.

Dans ce TP, nous utiliserons :

Élément	Valeur
GPIO utilisé	GPIO 17
Broche physique	11
Masse	GND

La commande suivante permet d'afficher le brochage du Raspberry Pi :

```
pinout
```

Attention Les GPIO du Raspberry Pi utilisent une logique de **3,3 V**.

Ne jamais appliquer directement une tension de 5 V sur une broche GPIO.

Une LED doit toujours être associée à une résistance de limitation de courant.

1.2 Câblage de la LED

Réaliser le montage suivant :

GPIO 17 (broche physique 11)

↓

Résistance 220 Ω

↓

LED

↓

GND

La broche GPIO fournit le signal de commande. Lorsque le GPIO est à l'état haut, la LED s'allume ; lorsqu'il est à l'état bas, elle s'éteint.

1.3 Vérifier le fonctionnement du GPIO

Raspberry Pi OS fournit l'outil `pinout`, associé à GPIO Zero.

Vérifier sa présence :

```
pinout
```

Repérer notamment :

- le numéro GPIO 17 ;
- le numéro de la broche physique correspondante ;
- les différentes broches GND.

2. Commander une LED avec Python

2.1 La bibliothèque GPIO Zero

GPIO Zero est une bibliothèque Python destinée à simplifier l'utilisation des GPIO du Raspberry Pi. Elle fournit des classes correspondant à des composants courants : LED, bouton poussoir, buzzer, moteur, capteur, etc.

Sur Raspberry Pi OS Desktop, GPIO Zero est normalement déjà installé.

Pour vérifier son installation :

```
python3 -c "import gpiozero; print(gpiozero.__version__)"
```

Si nécessaire, la bibliothèque peut être installée avec :

```
sudo apt update  
sudo apt install python3-gpiozero
```

2.2 Allumer une LED

Créer un fichier nommé `led.py` :

```
nano led.py
```

Écrire le programme suivant :

```
from gpiozero import LED  
  
led = LED(17)  
  
led.on()
```

Enregistrer le fichier puis l'exécuter :

```
python3 led.py
```

La LED doit s'allumer.

2.3 Éteindre la LED

Modifier le programme :

```
from gpiozero import LED  
  
led = LED(17)  
  
led.off()
```

Exécuter à nouveau :

```
python3 led.py
```

La LED doit s'éteindre.

2.4 Faire clignoter la LED

Créer un programme nommé `blink.py` :

```
from gpiozero import LED  
from time import sleep  
  
led = LED(17)  
  
while True:  
    led.on()  
    sleep(1)  
  
    led.off()  
    sleep(1)
```

Exécuter :

```
python3 blink.py
```

La LED doit s'allumer pendant une seconde puis s'éteindre pendant une seconde.

Le programme fonctionne en boucle infinie.

Pour l'arrêter :

```
CTRL + C
```

2.5 Analyse du programme

Identifier dans le programme :

- l'importation de la bibliothèque GPIO Zero ;
- la création de l'objet représentant la LED ;
- la broche GPIO utilisée ;
- la commande d'allumage ;
- la commande d'extinction ;
- la temporisation ;
- la boucle infinie.

2.6 Questions

1. Quelle est la différence entre le numéro GPIO et le numéro de la broche physique ?
2. Quelle instruction permet d'allumer la LED ?
3. Quelle instruction permet de l'éteindre ?
4. Que se passe-t-il si la valeur de `sleep(1)` est remplacée par `sleep(0.2)` ?
5. Modifier le programme afin que la LED reste allumée pendant 3 secondes puis éteinte pendant 1 seconde.

3. Variation de l'intensité lumineuse

Une sortie numérique possède deux états principaux :

- état bas : 0 ;
- état haut : 1.

Pour faire varier l'intensité lumineuse d'une LED, on peut utiliser une technique appelée **PWM** (Pulse Width Modulation, ou modulation de largeur d'impulsion).

GPIO Zero fournit la classe **PWMLED** pour ce type d'utilisation.

3.1 Programme

Créer le fichier `pwm.py` :

```
from gpiozero import PWMLED
from time import sleep

led = PWMLED(17)

while True:
    led.value = 0.0
    sleep(1)

    led.value = 0.25
    sleep(1)

    led.value = 0.5
    sleep(1)

    led.value = 0.75
    sleep(1)

    led.value = 1.0
    sleep(1)
```

Exécuter :

```
python3 pwm.py
```

Observer les variations de luminosité.

Arrêter le programme avec :

```
CTRL + C
```

3.2 Travail demandé

Modifier le programme afin de réaliser automatiquement un **fendu lumineux** :

1. augmentation progressive de la luminosité ;
2. luminosité maximale ;
3. diminution progressive de la luminosité ;
4. répétition du cycle.

4. Commander une LED depuis un navigateur

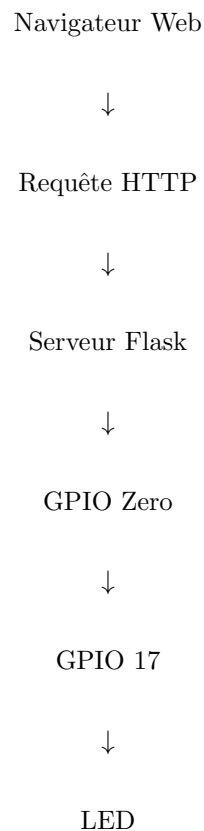
4.1 Principe

Nous allons maintenant ajouter une interface Web au programme.

L'objectif est de pouvoir commander la LED depuis un navigateur Web exécuté sur :

- le Raspberry Pi ;
- un ordinateur du même réseau ;
- une tablette ;
- un smartphone.

L'architecture du système sera la suivante :



4.2 Installation de Flask

Nous allons utiliser un environnement virtuel Python.

Créer un répertoire de travail :

```
mkdir ~/gpio-web  
cd ~/gpio-web
```

Installer les outils nécessaires :

```
sudo apt update  
sudo apt install python3-venv python3-gpiozero
```

Créer l'environnement virtuel :

```
python3 -m venv .venv --system-site-packages
```

Activer l'environnement virtuel :

```
source .venv/bin/activate
```

L'invite du terminal doit alors indiquer que l'environnement `.venv` est actif.

Installer Flask :

```
python -m pip install flask
```

Vérifier l'installation :

```
python -m pip show flask
```

4.3 Créer le serveur Web

Créer le fichier :

```
nano gpio_web.py
```

Entrer le programme suivant :

```
from flask import Flask, render_template_string, redirect, url_for
from gpiozero import LED

GPIO_PIN = 17

led = LED(GPIO_PIN)

app = Flask(__name__)

html_page = """
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Contrôle GPIO</title>
</head>
<body>
    <h1>Contrôle de la LED</h1>

    <p>GPIO utilisé : {{ gpio }}</p>

    <p>État actuel : <strong>{{ state }}</strong></p>

    <form action="{{ url_for('turn_on') }}" method="post">
        <button type="submit">Allumer</button>
    </form>

    <br>

    <form action="{{ url_for('turn_off') }}" method="post">
        <button type="submit">Éteindre</button>
    </form>
</body>
</html>
"""

@app.route("/")
def index():
    state = "ON" if led.is_lit else "OFF"
    return render_template_string(
        html_page,
        gpio=GPIO_PIN,
        state=state
    )
```

```
@app.route("/on", methods=["POST"])
def turn_on():
    led.on()
    return redirect(url_for("index"))

@app.route("/off", methods=["POST"])
def turn_off():
    led.off()
    return redirect(url_for("index"))

if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=8080
    )
```

4.4 Lancer le serveur

Vérifier que l'environnement virtuel est actif.

Si nécessaire :

```
cd ~/gpio-web
source .venv/bin/activate
```

Lancer le serveur :

```
python gpio_web.py
```

Flask indique alors que le serveur écoute sur le port 8080.

4.5 Tester depuis le Raspberry Pi

Ouvrir un navigateur Web sur le Raspberry Pi et saisir :

`http://localhost:8080`

Une page contenant deux boutons doit apparaître :

- Allumer
- Éteindre

Tester les deux boutons.

La LED doit s'allumer ou s'éteindre en fonction de l'action demandée.

4.6 Tester depuis un autre appareil

Le Raspberry Pi et l'appareil utilisé pour le test doivent être connectés au même réseau.

Pour connaître l'adresse IP du Raspberry Pi :

```
hostname -I
```

Par exemple, si l'adresse obtenue est :

```
192.168.1.42
```

ouvrir depuis l'autre appareil :

`http://192.168.1.42:8080`

Remplacer bien entendu l'adresse IP par celle obtenue sur votre Raspberry Pi.

4.7 Arrêter le serveur

Pour arrêter le serveur Flask :

```
CTRL + C
```

Puis, si nécessaire, quitter l'environnement virtuel :

```
deactivate
```

5. Travaux Pratiques

5.1 TP 1 : Commander une LED depuis un navigateur

Objectif

Réaliser une application permettant de commander une LED connectée au Raspberry Pi 5 depuis un navigateur Web.

L'application devra respecter l'architecture suivante :

Navigateur → Flask → GPIO Zero → GPIO 17 → LED

Travail demandé

1. Réaliser le câblage de la LED sur le GPIO 17.
2. Vérifier le brochage du Raspberry Pi avec :

```
pinout
```

3. Créer le répertoire du projet :

```
mkdir ~/gpio-web  
cd ~/gpio-web
```

4. Créer et activer un environnement virtuel Python.
5. Installer Flask.
6. Créer le programme `gpio_web.py`.
7. Lancer le serveur Web.
8. Tester l'application depuis le Raspberry Pi.
9. Déterminer l'adresse IP du Raspberry Pi avec :

```
hostname -I
```

10. Tester l'application depuis un autre équipement connecté au même réseau.
11. Vérifier que les boutons de l'interface Web commandent effectivement la LED.

5.2 Amélioration 1 : ajouter un bouton de clignotement

Modifier l'application afin d'ajouter un troisième bouton :

Clignoter

Lorsque l'utilisateur clique sur ce bouton, la LED doit clignoter plusieurs fois.

5.3 Amélioration 2 : modifier le temps de clignotement

Ajouter dans le programme une variable permettant de régler la durée entre deux changements d'état.
Par exemple :

```
TEMPO = 0.5
```

Modifier cette valeur et observer son influence sur le fonctionnement de la LED.

5.4 Amélioration 3 : afficher l'état de la LED

L'interface Web doit indiquer clairement :

- **LED : ÉTEINTE**
- ou
- **LED : ALLUMÉE**

L'état affiché doit correspondre à l'état réel de la LED.

6. Questions de synthèse

1. Quel est le rôle d'un GPIO ?
2. Quelle est la différence entre une entrée GPIO et une sortie GPIO ?
3. Quel est le numéro GPIO utilisé dans ce TP ?
4. Quel est le numéro de la broche physique correspondante ?
5. Pourquoi faut-il utiliser une résistance avec une LED ?
6. Quel est le rôle de GPIO Zero ?
7. Quel est le rôle de Flask ?
8. Quel protocole réseau est utilisé entre le navigateur et le serveur Flask ?
9. Pourquoi le serveur Flask utilise-t-il le port 8080 ?
10. Quelle commande permet de connaître l'adresse IP du Raspberry Pi ?
11. Quelle est la différence entre :
 - `localhost:8080` ;
 - `IP_DU_RPI:8080` ?
12. Représenter sous forme de schéma le chemin parcouru par l'information lorsque l'utilisateur clique sur le bouton **Allumer**.